# Labo 16 — Antwoorden met toelichting

*Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, een voorbeeld dat je in de terminal kunt nagaan.*

---

### Vraag 1 — Requests en limits

**Antwoord.** De **request** wordt gelezen door de **scheduler**: hij kiest een node waarvan de resterende capaciteit (capaciteit min de som van de al geplaatste requests) de vraag dekt. De **limit** wordt door de **kubelet** in de **cgroup** van de container geschreven. Geheugen boven de limit: de kernel doodt het proces (`OOMKilled`, code 137), de kubelet herstart (`RESTARTS` + 1, daarna `CrashLoopBackOff`). CPU boven de limit: de cgroup **vertraagt** de container (*throttling*), niets wordt gedood.

**Waarom.** Geheugen wordt niet "uitgeleend": een toegewezen pagina blijft bezet tot ze vrijgegeven wordt; CPU wordt in tijdschijven gedeeld.

**Nuance.** De CPU-request dient ook voor de evenredige verdeling (`cpu.weight`) als de node verzadigd is; de geheugen-request dient voor de berekening van de verwijdering. Zonder geheugenlimit kan een container de node leegzuigen en de verwijdering van andere Pods veroorzaken.

**Voorbeeld.**

```
kubectl describe node | sed -n '/Allocated resources/,/Events/p'   # Requests  Limits  per resource
kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'   # OOMKilled
```

---

### Vraag 2 — QoS en verwijderingsvolgorde

**Antwoord.** `db` is `Guaranteed` (requests = limits), `api` is `Burstable`, `batch` is `BestEffort`. Bij geheugendruk verwijdert de kubelet eerst de `BestEffort` (`batch`), dan de `Burstable` die hun request overschrijden (`api`), en de `Guaranteed` als laatste. Als `batch` een kritieke verwerking was, zou hij als eerste geofferd worden, gewoon omdat niemand gedeclareerd heeft wat hij verbruikt.

**Waarom.** De kubelet heeft geen enkel besef van zakelijk belang; hij kent alleen wat de spec declareert. Een Pod zonder `resources` is per definitie een Pod die gemist kan worden.

**Nuance.** De QoS regelt alleen de verwijdering door de **kubelet**; de OOM killer van de kernel handelt per cgroup wanneer een limit overschreden wordt, ongeacht de klasse. Voor een belangrijke batch: requests = limits, of minstens realistische requests, en een `PriorityClass` als het platform er aanbiedt.

**Voorbeeld.**

```
kubectl get pods -o custom-columns=NAME:.metadata.name,QOS:.status.qosClass
```

---

### Vraag 3 — `OOMKilled`

**Antwoord.** `kubectl describe pod glouton` toont `Reason: OOMKilled`, `Exit Code: 137` en de limit (`Limits: memory: 32Mi`); `kubectl get pod glouton -o jsonpath='{.status.containerStatuses[0].lastState.terminated.reason}'` geeft `OOMKilled`. `RESTARTS` stijgt omdat `restartPolicy: Always` (standaard) de kubelet de container na elke dood laat herstarten, met een groeiende wachttijd (`CrashLoopBackOff`). Hier is de applicatie ontworpen om 100 MB toe te wijzen met een limit van 32 Mi: beide moeten herzien worden — maar in het echte leven is het bijna altijd **de applicatie** (lek, slecht gedimensioneerde JVM-heap) of een **te krappe limit**, niet de limit die "op zich te laag" is.

**Waarom.** 137 = 128 + 9: het proces kreeg `SIGKILL` van de kernel (labo 03, labo 10).

**Nuance.** Een `OOMKilled` kan ook een container **zonder** limit treffen: als de hele node geheugen tekortkomt, doodt de kernel het grootste proces. `describe node` toont dan `MemoryPressure True`.

**Voorbeeld.**

```
kubectl describe pod glouton | grep -E 'Reason:|Exit Code|Restart Count'
#       Reason:       OOMKilled  /  Exit Code:    137  /  Restart Count:  1
```

---

### Vraag 4 — Drie Pods `Pending`

**Antwoord.** De scheduler vindt geen node meer waarvan het **reserveerbare** geheugen nog één request dekt; de Pods blijven `Pending` met `FailedScheduling: 0/1 nodes are available: 1 Insufficient memory`. Mogelijke correcties: (1) **de request verlagen** van de API als ze overschat is (meten met `kubectl top pods`); (2) **een node toevoegen** of geheugen aan de node geven (`.wslconfig` op WSL); (3) de node **ontlasten** van andere Pods (of de replica's verminderen). De eerlijkste is (1) **als** de meting toont dat de API ruim onder haar request draait; anders is het (2): een realistische request is geen instelling, het is een feit.

**Waarom.** De plaatsing gebeurt op de **requests**, niet op het werkelijke verbruik: een node kan "vol" zijn met 95 % requests bij 20 % gebruik.

**Nuance.** De request onder het werkelijke gebruik zetten maakt van de Pod een overconsumerende `Burstable`: hij wordt bij druk als eerste verwijderd. Een beheerde cluster voegt automatisch een node toe (*cluster autoscaler*); k3s op WSL niet.

**Voorbeeld.**

```
kubectl describe pod trop-gourmand | grep FailedScheduling
#  Warning  FailedScheduling  … 0/1 nodes are available: 1 Insufficient memory. …
```

---

### Vraag 5 — LimitRange

**Antwoord.** (a) De Pod krijgt `requests: {memory: 32Mi}` en `limits: {memory: 128Mi}` bij de creatie in zijn spec ingevoegd (zichtbaar in `kubectl get pod -o yaml`). (b) De Pod wordt bij de `apply` **geweigerd**: `maximum memory usage per Container is 1Gi, but limit is 4Gi`. (c) De Pod behoudt zijn request van `64Mi` en krijgt standaard `limits.memory: 128Mi`. Het platformteam legt deze objecten op zodat geen enkele Pod zonder gedeclareerde resources draait (dus in `BestEffort`) en geen enkele een volledige node kan reserveren.

**Waarom.** De LimitRange werkt als een **admission controller**: hij wijzigt (standaardwaarden) of verwerpt (max/min) elke Pod op het moment van zijn creatie.

**Nuance.** De LimitRange raakt al bestaande Pods niet; en als `defaultRequest` groter is dan `default`, is het object ongeldig. Een `default` zonder `defaultRequest` geeft request = limit (dus `Guaranteed`).

**Voorbeeld.**

```
kubectl run sans-limites --image=docker.io/library/alpine --command -- sleep 3600
kubectl get pod sans-limites -o jsonpath='{.spec.containers[0].resources}'
# {"limits":{"cpu":"250m","memory":"128Mi"},"requests":{"cpu":"50m","memory":"32Mi"}}
```

---

### Vraag 6 — ResourceQuota overschreden

**Antwoord.** Het resterende budget `requests.memory` is `1Gi − 3×64Mi = 832Mi`; acht Pods van 128Mi vragen er 1024Mi. De **ReplicaSet** maakt de Pods een voor een: zes gaan door (768Mi), de zevende wordt geweigerd. `kubectl get pods` toont **zes** Pods van het tweede Deployment, geen acht; `kubectl describe deployment` zegt `6/8` (`ReplicaFailure True`), en de weigering lees je in **`kubectl describe replicaset`** (of `kubectl get events`): `exceeded quota: budget, requested: requests.memory=128Mi, used: 896Mi, limited: 1Gi`.

**Waarom.** De quota wordt gecontroleerd bij de creatie van elke **Pod** door de API server; het Deployment zelf wordt aanvaard — het verbruikt niets.

**Nuance.** De grens `pods: 10` zou ook meespelen: 3 + 8 = 11 > 10, dus hoe dan ook hoogstens zeven Pods. De ReplicaSet blijft eindeloos proberen: zodra er budget vrijkomt, verschijnen de ontbrekende Pods.

**Voorbeeld.**

```
kubectl describe quota budget | sed -n '/Resource/,$p'
# requests.memory  160Mi  1Gi  …
```

---

### Vraag 7 — `logs --previous`

**Antwoord.** `kubectl logs` toont de **huidige** container: na een herstart is dat een nieuw proces, waarvan de logs bij de opstart beginnen. De doodsoorzaak zit in de uitvoer van de **vorige** container, die de kubelet één generatie lang bewaart: `--previous`. `lastState` toont `{"terminated":{"reason":"OOMKilled"|"Error"|"Completed","exitCode":137,"finishedAt":…}}`: de reden en de code van de laatste gestorven container.

**Waarom.** Elke herstart maakt een nieuwe container; de runtime bewaart de vorige (en zijn logboek) tot de volgende.

**Nuance.** Twee herstarts later is het eerste logboek verloren: vandaar de gecentraliseerde logverzameling (Loki, Elastic) in productie. En `--previous` faalt op een Pod die nooit herstart is.

**Voorbeeld.**

```
kubectl exec $POD -- sh -c 'kill 1' ; sleep 5
kubectl logs $POD --previous | tail -1     # API arretee proprement.
```

---

### Vraag 8 — `kubectl debug --target`

**Antwoord.**

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api --profile=general -- sh
```

`--target=api` plaatst de efemere container in de **proces-namespace** van de container `api`: `ps` toont de `java -jar`, `/proc/1/root/` geeft toegang tot zijn bestandssysteem, en `localhost:8080` is zijn poort (netwerk-namespace van de Pod). Dat is beter dan een shell in de image omdat de image minimaal en zonder tools voor een aanvaller blijft (labo 05), en omdat de diagnoseshell gekozen wordt **op het moment** van de behoefte, met de juiste tools (`netshoot` voor het netwerk, `alpine` voor een `ps`).

**Waarom.** Efemere containers zijn een veld `ephemeralContainers` in de spec van de Pod, toegevoegd zonder de Pod te herstarten; ze kunnen niet verwijderd worden, maar verdwijnen met hem.

**Nuance.** Zonder `--target` deelt de efemere container het netwerk maar niet de processen. `kubectl debug node/wsl-ubuntu` opent op dezelfde manier een shell op de node. Een boodschap `--profile=legacy is deprecated` verschijnt als je `--profile` weglaat.

**Voorbeeld.**

```
kubectl debug $POD -it --image=docker.io/library/alpine --target=api -- sh -c 'ps aux | head
-3; wget -qO- localhost:8080/actuator/health'
#     1 1000      0:01 java -jar /app/api.jar   …   {"status":"UP"}
```

---

### Vraag 9 — `Forbidden`

**Antwoord.** `Error from server (Forbidden)`: de API server heeft de aanroeper geauthenticeerd en daarna de autorisatie geweigerd. `User "system:serviceaccount:lab16:reader"`: het subject is de ServiceAccount `reader` van de namespace `lab16` (prefix `system:serviceaccount:`). `cannot list resource "secrets" in API group ""`: het werkwoord `list`, de resource `secrets`, van de *core*-groep (leeg). `in the namespace "lab16"`: de geteste reikwijdte. Dit is **het gewenste gedrag**: de Role somt alleen `pods` en `pods/log` op. Voor de ConfigMaps: een regel toevoegen met `resources: ["configmaps"], verbs: ["get", "list", "watch"]` — en vooral geen `secrets`.

**Waarom.** RBAC is **additief en standaard gesloten**: alles wat niet expliciet toegekend is, wordt geweigerd.

**Nuance.** `--as` vereist het recht `impersonate`, dat `cluster-admin` heeft; daardoor kun je testen zonder een kubeconfig aan te maken. De meegeleverde ClusterRole `view` sluit de Secrets ook uit, met opzet.

**Voorbeeld.**

```
kubectl auth can-i get secrets --as=system:serviceaccount:lab16:reader    # no
kubectl auth can-i get pods    --as=system:serviceaccount:lab16:reader    # yes
```

---

### Vraag 10 — Role versus ClusterRole

**Antwoord. Role + RoleBinding**: rechten op de resources **van één namespace** (het facturatieteam mag deployen in `facturatie-prod`). **ClusterRole + ClusterRoleBinding**: rechten **op de hele cluster**, inclusief de niet-namespaced resources (nodes, PV's, namespaces): het platformteam, een monitoringagent die alle Pods leest. **ClusterRole + RoleBinding**: een rol die **één keer** op clusterniveau gedefinieerd is, maar **in één namespace** toegekend wordt: zo geef je

`edit` op `facturatie-prod` zonder ze overal opnieuw te definiëren. De ClusterRoles `view` / `edit` / `admin` bestaan precies om via RoleBinding namespace per namespace gekoppeld te worden.

**Waarom.** Een RoleBinding beperkt de gerefereerde rol altijd tot zijn eigen namespace, ongeacht het type van de rol.

**Nuance.** `admin` via RoleBinding laat toe de RBAC van de namespace te beheren, maar niet om namespaces te maken of aan de quota's te raken. `cluster-admin` via ClusterRoleBinding is de sleutel tot alles: voorbehouden aan de mensen van het platform, nooit aan een applicatieve ServiceAccount.

**Voorbeeld.**

```
kubectl create rolebinding dev-edit --clusterrole=edit --user=alice -n facturatie-prod
kubectl auth can-i create deployments --as=alice -n facturatie-prod   # yes
kubectl auth can-i create deployments --as=alice -n default           # no
```

---

### Vraag 11 — NetworkPolicy

**Antwoord.** Eerste `wget`: mislukt (`wget: can't connect to remote host (10.43.x.x): Connection refused`, of *timeout*). Tweede, na het label `app=web`: `{"status":"UP"}`. Zodra een NetworkPolicy een Pod **selecteert** (`podSelector: app=api`), wordt al het inkomende verkeer dat niet opgesomd is geweigerd; daarvoor, zonder policy, ging alles door. Op een cluster waarvan de CNI de policies niet toepast, wordt het object door de API aanvaard en **genegeerd**: beide `wget`'s slagen, zonder enige foutmelding.

**Waarom.** De NetworkPolicy is declaratief; het is de CNI (kube-router in k3s) die ze vertaalt naar filterregels op de node, en die de labels voortdurend leest — vandaar het onmiddellijke effect van een `kubectl label`.

**Nuance.** De policy kijkt alleen naar Pods: het ClusterIP wordt vertaald vóór het filteren, en het bron-IP is dat van de client-Pod. De weigering uit zich naargelang de CNI als een `refused` of als stilte (`timeout`); test altijd met `--timeout`.

**Voorbeeld.**

```
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   # mislukt
kubectl label pod client app=web
kubectl exec client -- wget -qO- --timeout=3 http://api:8080/actuator/health   #
{"status":"UP"}
```

---

### Vraag 12 — Policies voor de stack

**Antwoord.** Twee `ingress`-policies: (1) `podSelector: app=db`, `from: podSelector app=api`, poort 5432; (2) `podSelector: app=api`, `from:` twee bronnen, `podSelector app=web` **en** Traefik, poort 8080. De moeilijkheid: Traefik draait in `kube-system`, en een `podSelector` alleen kijkt enkel in de huidige namespace; je hebt een `namespaceSelector` nodig (label `kubernetes.io/metadata.name: kube-system`) **gecombineerd** met de `podSelector` van Traefik in **hetzelfde** element van `from` (twee aparte elementen zouden een OF vormen). Men begint met een *deny-all*-policy (`podSelector: {}` zonder `ingress`) omdat ze de weigering tot **standaard** maakt: een nieuwe Pod is alleen bereikbaar als iemand dat beslist heeft.

**Waarom.** Zonder policy die hem selecteert, blijft `web` voor iedereen bereikbaar — gewild, want hij wordt via de Ingress bediend; maar elke andere Pod die morgen toegevoegd wordt, zou open staan.

**Nuance.** Een *deny-all* in `egress` snijdt ook de **DNS** af: je moet `kube-system`/CoreDNS toelaten op 53 UDP/TCP, een klassieke valkuil. En de probes van de kubelet zijn niet betrokken: ze komen van de node, niet van een Pod.

**Voorbeeld.**

```
ingress:
  - from:
      - namespaceSelector: { matchLabels: { kubernetes.io/metadata.name: kube-system } }
        podSelector: { matchLabels: { app.kubernetes.io/name: traefik } }
      - podSelector: { matchLabels: { app: web } }
    ports: [ { port: 8080 } ]
```

---

### Vraag 13 — Gestolen token

**Antwoord.** Met `default` zonder RoleBinding: het token is geldig (het authenticeert) maar **geeft geen enkel recht**; de aanvaller krijgt hoogstens `kubectl auth can-i --list` en enkele anonieme lezingen (`/version`, discovery). Met `default` gekoppeld aan `cluster-admin`: hij krijgt **de hele cluster**: alle Secrets van alle namespaces lezen, een geprivilegieerde Pod maken op om het even welke node, dus de node zelf. De twee maatregelen van het labo: een **eigen ServiceAccount zonder rechten** (en `automountServiceAccountToken: false`, aangezien de API de API server niet aanroept), en een `egress`-**NetworkPolicy** die de Pod belet iets anders te bereiken dan wat hij moet (de API server inbegrepen, als niets dat rechtvaardigt).

**Waarom.** Het token is een bestand dat het proces kan lezen (`/var/run/secrets/kubernetes.io/serviceaccount/token`); alles wat het proces kan, kan de aanvaller die het uitvoert ook.

**Nuance.** Gemounte tokens zijn tegenwoordig **geprojecteerd** en beperkt in tijd (een uur, vernieuwd door de kubelet), wat een exfiltratie beperkt; maar een token dat een uur geldig is met `cluster-admin` volstaat ruimschoots. `kubectl get clusterrolebindings -o wide | grep cluster-admin` is de basisaudit.

**Voorbeeld.**

```
kubectl get clusterrolebindings -o wide | grep -w cluster-admin       # wie is root van de cluster?
kubectl get pod $POD -o jsonpath='{.spec.serviceAccountName}'         # default?
```